Risk Analysis Report

Team 7

2351441 许奕

2351039 王相

2350283 康凤轩

2352746 张洛梧

2350217 陈奕玮

Risk Analysis Report

1. Purpose, Scope, and Object Under Analysis

This report is the **risk analysis report for the chosen target application**, not for the AutoTestDesign tool itself.

The target application under test is **MiniShop Checkout**, a compact e-commerce checkout prototype implemented in:

- `target_app/minishop_checkout/`

Its formal application definition is documented in:

- `final_docs/12_target_application_definition_cn.md`

The final project uses `ARG-Test` as the AutoTestDesign tool to analyze requirements and generate structured test artifacts, but the object of risk analysis in this report is:

- **MiniShop Checkout as the application under test**

Within the final submission:

- `ARG-Test` is the tool
- `MiniShop Checkout` is the target application
- `coupon_discount_engine` is the selected major module of `MiniShop Checkout` used in the detailed test design and execution document

The scope of this report covers the parts of `MiniShop Checkout` that can directly affect order preview correctness, payment acceptance, pickup correctness, or final customer-facing totals:

- cart subtotal preparation
- promotion and coupon logic
- shipping fee calculation
- tax and order-total calculation
- payment-card validation
- pickup-station and recipient validation
- checkout orchestration across the above components

This report therefore answers the question:

> If MiniShop Checkout were the application we needed to validate for this course project, where is the greatest defect risk, why is it risky, and where should the testing effort go first?

---

2. Application Context and Business Flow

`MiniShop Checkout` is intentionally compact, but it still contains enough decision-heavy behavior to justify systematic testing. Its job is not only to add prices together. It must coordinate several types of rules:

- monetary thresholds
- mutually exclusive promotions
- input validation rules
- fulfillment-mode-specific validation
- country-specific tax treatment
- cross-component consistency in the final order preview

The high-level business flow of the application is:

1. calculate the merchandise subtotal from cart items
2. calculate a shipping quote from destination, weight, method, and subtotal
3. apply coupon logic on top of the subtotal and shipping context
4. calculate tax from the discounted subtotal and shipping fee
5. validate payment data if a card is provided
6. validate pickup data if fulfillment mode is pickup
7. combine the above outputs into one final checkout preview

This means the application can fail in at least three distinct ways:

- **local rule failure**
  - a single component computes a wrong result
- **validation failure**
  - invalid input is accepted, or valid input is rejected
- **integration-consistency failure**
  - individually plausible component outputs are combined into an incorrect final preview

The major components used in this risk report are:

Application component	Main responsibility	Main code path	Main risk character
Cart and subtotal preparation	aggregate item-level values into merchandise subtotal	`target_app/minishop_checkout/models.py`, `target_app/minishop_checkout/checkout.py`	arithmetic and item-data integrity
Promotion service	validate coupon eligibility and compute discount effect	`target_app/minishop_checkout/promotion.py`, `reference_impl/coupon_discount_engine.py`	rule combination, thresholds, financial leakage
Shipping service	calculate fee by zone, method, weight, and free-shipping threshold	`target_app/minishop_checkout/shipping.py`	tier logic, unsupported combinations, threshold mistakes
Tax service	calculate taxable amount, tax, and final total	`target_app/minishop_checkout/tax.py`	formula correctness, country-dependent behavior, rounding
Payment validation service	validate cardholder name, card number, expiry, brand, and CVV	`target_app/minishop_checkout/payment.py`	valid/invalid partition handling
Pickup validation service	validate station ID, phone, pickup code, and note	`target_app/minishop_checkout/pickup.py`	format validation and conditional requiredness
Checkout orchestration	assemble all component outputs into one preview	`target_app/minishop_checkout/checkout.py`	cross-component consistency

---

3. Risk Analysis Method

3.1 Scoring Dimensions

Each risk is evaluated on three `1-5` scales:

- `Impact`
  - how severe the business or user-facing consequence is if the defect escapes
- `Likelihood`
  - how likely defects are, based on rule density, boundaries, interactions, and implementation complexity
- `Detectability`
  - how hard the defect is to discover without deliberate and systematic test design

The final score is:

```
Risk Priority = Impact x Likelihood x Detectability
```

3.2 Priority Bands

- `High`: `>= 60`
- `Medium`: `36-59`
- `Low`: `<= 35`

These bands are intentionally simple because the project is course-scale. The main purpose is prioritization, not pretending to produce a production-grade quantitative safety model.

3.3 Evidence Inputs Used for Scoring

The scoring in this report is derived from the combination of:

- target-application code paths in `target_app/minishop_checkout/`
- the selected detailed module in `reference_impl/coupon_discount_engine.py`
- target-application smoke tests in `tests/test_minishop_checkout_smoke.py`
- selected detailed-module tests in:
  - `tests/test_coupon_discount_engine_blackbox.py`
  - `tests/test_coupon_discount_engine_whitebox.py`
- the target-application definition in `final_docs/12_target_application_definition_cn.md`
- the NFR evidence package in:
  - `final_docs/execution_evidence/nfr_validation_summary.md`

3.4 Risk-Scoring Assumptions

The following assumptions are used consistently in this report:

1. financial miscalculation risks are treated as high-impact even in a classroom prototype because they directly affect order totals
2. acceptance/rejection mistakes in payment and pickup validation are treated as operationally important because they block checkout or allow bad input through
3. components with multiple thresholds, interacting rules, or cross-field constraints receive higher likelihood and detectability scores
4. a component that currently has only smoke-level executable evidence retains a higher residual risk than the selected detailed module, even if its implementation is small

---

4. Component-Level Risk Drivers

Before listing the detailed register, it is useful to summarize *why* each component is risky.

Component	Main risk driver	Why the risk is non-trivial
Cart subtotal	item-level arithmetic and data integrity	wrong subtotal poisons every downstream step
Promotion	multiple interacting rules	coupon logic mixes thresholds, exclusivity, premium-only logic, sale-item restrictions, and explicit rejection behavior
Shipping	tier resolution and method eligibility	zone, weight, method, and subtotal interact
Tax	country-specific formula and rounding	taxable base changes by country and rounding must stay consistent
Payment	multi-field validation	the result depends on name, card digits, expiry window, brand, and CVV together
Pickup	conditional requiredness	pickup code is optional in one mode but mandatory in another
Checkout orchestration	cross-component consistency	final totals and validation notes depend on several component outputs remaining aligned

This component-level view already suggests that the most dangerous areas are not the simple single-field checks, but:

- promotion
- tax/total calculation
- payment validation
- orchestration

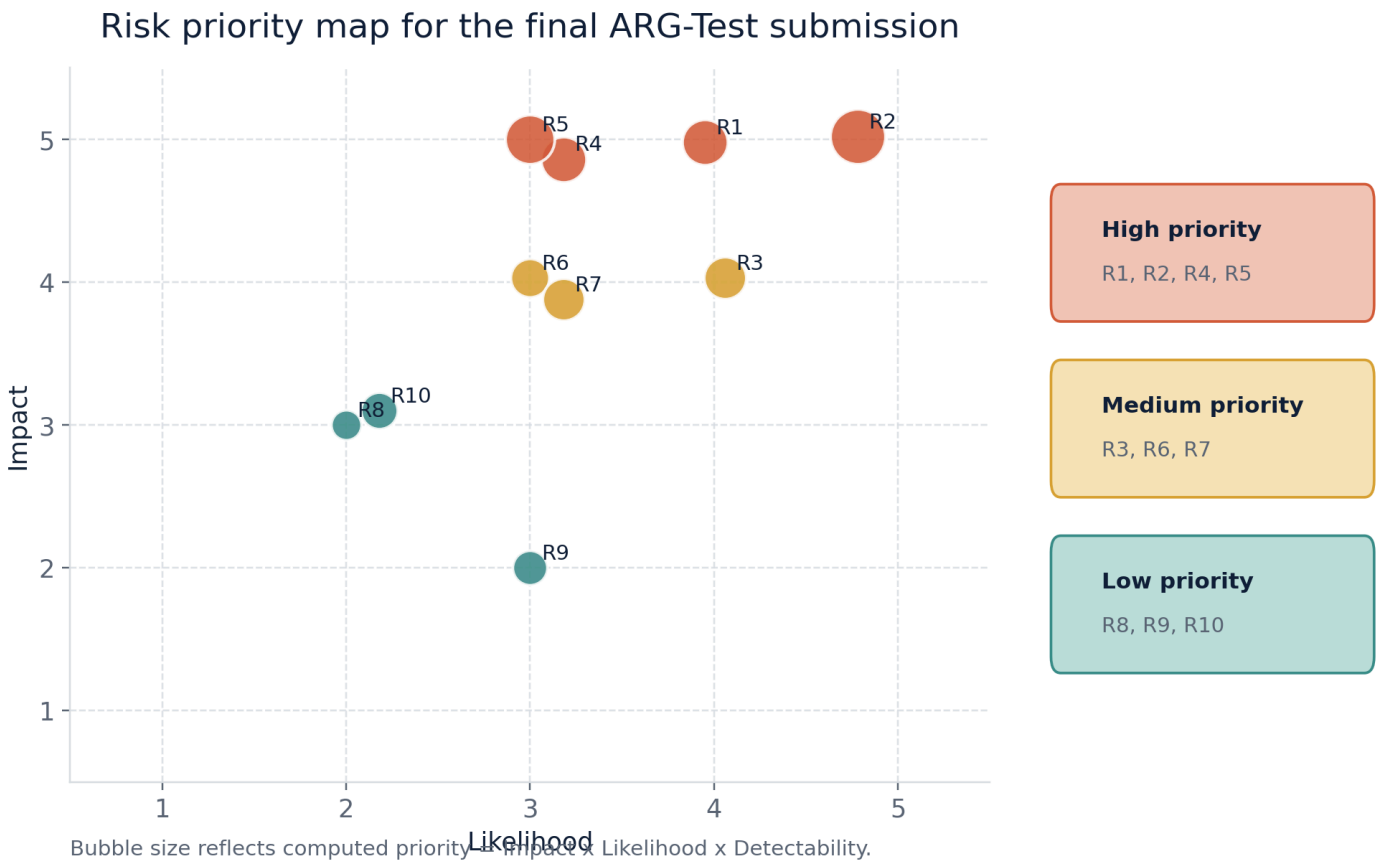
---

5. Detailed Risk Register

Risk ID	Component / feature	Requirement basis	Concrete failure event	I/L/D	Priority	Band	Preferred test focus	Current mitigation evidence	Resid risk
R1	Promotion discount and coupon logic	coupon_discount_engine	wrong discount amount, wrong coupon acceptance, financial leakage, missed exclusivity or threshold rule	5/5/4	100	High	Decision Table + EP + BVA + white-box	strongest evidence exists here: 15 module tests, full statement/branch coverage, mutation demo	Medi
R2	Shipping fee calculation and shipping-method eligibility	shipping_fee_calculator	wrong tier fee, free shipping granted incorrectly, express shipment accepted when overweight	4/4/4	64	High	BVA + Decision Table	smoke-level executable support and code reviewable structure	Medi High
R3	Tax and order-total calculation	order_total_tax_calculation	wrong taxable base, wrong country-rate application, wrong rounding, wrong final order total	5/4/4	80	High	BVA + Decision Table + calculation spot checks	concrete implementation exists and is reachable through smoke preview	High
R4	Payment-card validation	payment_card_expiry_and_cvv_validation	invalid card accepted,	5/4/4	80	High	EP + BVA	target-app implementation	Medi High

Risk ID	Component / feature	Requirement basis	Concrete failure event	I/L/D	Priority	Band	Preferred test focus	Current mitigation evidence	Resid risk
			valid card rejected, current-month expiry logic handled incorrectly					and smoke validation path exist	
R5	Pickup-station and recipient validation	`pickup_station_contact_validation`	invalid station, phone, or pickup code accepted; valid pickup input blocked	4/3/4	48	Medium	EP + BVA	validation logic and smoke-level evidence exist	Medi
R6	Checkout orchestration and cross-component consistency	integration in `checkout.py`	final total inconsistent with coupon, shipping, and tax outputs; validation note omitted; wrong note attached	5/4/4	80	High	scenario-based integration checks	end-to-end smoke scenarios already exist	High
R7	Cart subtotal preparation	item-level arithmetic in `checkout.py` and `models.py`	item discount exceeds gross value, negative item data, subtotal propagated incorrectly to later stages	4/3/4	48	Medium	EP + arithmetic guard checks	implementation has defensive checks but less dedicated execution evidence than coupon module	Medi
R8	Performance of checkout preview	NFR performance interpretation for classroom-scale usage	preview path becomes slow enough that iterative review and repeated test runs become impractical	3/2/3	18	Low	NFR performance measurement	latest local/mock NFR run processes `100` requirements in `0.3646 s`	Low
R9	Validation-message usability and maintainability	NFR usability/maintainability	users receive unclear correction guidance or future rule changes become hard to verify consistently	3/3/3	27	Low	smoke checks, review of notes/messages, modular-code review	small modular codebase, Web demo review path, structured artifacts, `45` passing tests	Low-Medi

6. Risk Heatmap and Priority Interpretation



6.1 Highest-Priority Risks

The most important risks are R1, R3, R4, and R6.

They deserve the most attention for different reasons:

- R1 Promotion logic
  - high financial impact
  - multiple interacting rules
  - compact enough to justify full detailed execution
- R3 Tax and total calculation
  - easy to overlook because formula mistakes may still look numerically plausible
  - directly affects customer payment amount
- R4 Payment validation
  - defects become immediate user blockers or input-quality failures
  - includes multiple validity rules with time-sensitive behavior
- R6 Checkout orchestration
  - a correct component result can still be presented incorrectly at the integration level

6.2 Medium-Priority Risks

R2, R5, and R7 remain meaningful because they contain real validation and tier logic, but they currently have a narrower blast radius than the top financial and integration risks.

These risks still require planned suites and executable evidence, but not necessarily the same white-box depth as the coupon module.

6.3 Lower-Priority but Still Relevant Risks

R8 and R9 are not ignored just because they are lower than the financial rules.

They remain relevant because the assignment explicitly asks for:

- performance
- usability / UX / UI
- maintainability and technology

The correct course-project interpretation is therefore:

- they are not the first debugging priority
- but they must still appear in the validation package and in the test plan

---

7. Risk-Driven Test Strategy

The risk register directly determines where the test effort should go.

Priority group	Risks	Planned suite focus	Main techniques	Reason for selection
Group A	R1, R3, R4, R6	promotion suite, tax/total suite, payment suite, orchestration suite, detailed module suite	Decision Table, EP, BVA, scenario integration, selected white-box evidence	highest business impact and strongest interaction density
Group B	R2, R5, R7	shipping suite, pickup suite, subtotal guard checks	EP, BVA, rule-table spot checks	clear threshold and format logic, but lower blast radius than top financial risks
Group C	R8, R9	NFR and traceability suite	performance measurement, usability review, maintainability evidence	assignment-required but less urgent than pricing/payment correctness

This means the test strategy should not distribute effort evenly. It should front-load:

1. promotion logic
2. payment and tax correctness
3. checkout integration consistency

and only then allocate lighter effort to:

4. pickup validation
5. shipping edge cases not already covered by main scenarios
6. course-scale NFR evidence

---

8. Mitigation Status and Residual Risk

8.1 Strongest Current Mitigation

The strongest current mitigation is the selected detailed module:

- `coupon_discount_engine`

Why it matters:

- it sits inside the highest-risk promotion area
- it already has multiple black-box techniques represented
- it has executable white-box evidence
- it has full statement and branch coverage
- it has `4 / 4` mutant kills

This significantly lowers the residual risk of `R1`, even though it does not reduce it to zero.

8.2 Risks Still Not Mitigated to the Same Depth

The following risks still have higher residual uncertainty because they currently rely more on structure plus smoke-level evidence:

- `R2` shipping
- `R3` tax and total calculation
- `R4` payment validation
- `R6` orchestration consistency

This does not mean these areas are untested. It means:

- they are represented in the application
- they are covered in the test plan
- but they do not yet have the same detailed module-level execution depth as `coupon_discount_engine`

8.3 Recommended Next Mitigation Order

If the team had one more iteration beyond the current deliverables, the most valuable next work would be:

1. add deeper executable tax/total checks
2. add more explicit payment-expiry boundary checks at current-month and future-window edges
3. add more orchestration scenarios that combine invalid payment or pickup data with otherwise valid financial calculations

---

9. Relation to ARG-Test

`ARG-Test` supports the testing of `MiniShop Checkout` by:

1. ingesting application requirements
2. structuring them into auditable traces
3. assigning risk and test priority

4. generating black-box suites with explicit technique obligations
5. exporting reviewable artifacts
6. supporting designer-in-the-loop review and revised-suite export

However, this report remains an **application-risk** report.

The correct interpretation is:

- risk scoring here prioritizes the testing effort for `MiniShop Checkout`
- `ARG-Test` is the design-support tool
- the selected coupon module is the strongest execution anchor inside the chosen target application

---

10. Honest Boundaries

The following boundaries should stay explicit:

- `MiniShop Checkout` is a compact course-project prototype, not a production checkout platform
- only `coupon_discount_engine` currently has full black-box, white-box, coverage, and mutation evidence
- the other application components currently provide concrete structure and smoke-level executable support rather than equally deep white-box analysis
- the repository still contains a broader requirement corpus used to evaluate `ARG-Test` as a tool; those extra requirements should not be confused with the formal application scope of `MiniShop Checkout`

These boundaries do not weaken the project as long as the final submission states them accurately.

---

11. Conclusion

`MiniShop Checkout` is a concrete checkout-oriented target application whose highest-risk areas are:

- promotion and coupon logic
- tax and order-total calculation
- payment-card validation
- checkout orchestration

The final conclusions of this report are:

1. the target application has several **High** priority areas that justify systematic testing rather than nominal-path checking
2. `coupon_discount_engine` is the strongest selected high-risk module for detailed executable evidence
3. the test plan should allocate the most effort to promotion, tax/payment correctness, and orchestration consistency
4. NFR concerns are lower than financial-rule defects, but they are still present and are already supported by explicit evidence

This interpretation aligns the report with the assignment requirement that the risk analysis must target the **application under test**, not the AutoTestDesign tool itself.